

SOFTWARE TESTING · AI-FIRST · REDESIGNED

API Testing — Requirements, Techniques & Examples

Functional requirements make the API work. Non-functional requirements make it production-ready. For each — the test technique and a concrete example.

Fundamentals

Functional + contract

9 NFRs

Strategy · CI/CD · AI

SUT: EShop API · examples in Postman / REST Assured / k6 · OWASP API Security Top 10 (2023)

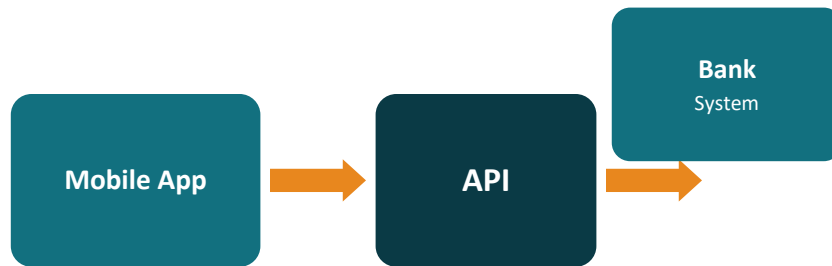
What is an API?

An interface that lets two software systems talk — requests in, structured responses out.

API = Application Programming Interface.

It defines the requests you can make, how to make them, and the data formats exchanged — between two systems, with no GUI in between.

Testing targets the business-logic layer directly — not the look & feel.



Request → GET /accounts/42/balance

Response ← { balance: 1 500 000, currency: 'VND' }

EXAMPLE SCENARIO “Get Account Balance” — the API must return the current balance. That's the functional requirement.

What is API testing?

Drive the API with software calls; assert the response — status, body, schema, timing.

- › **Checks functionality, reliability, performance & security** — of the interface itself.
- › **No GUI, no keyboard/mouse** — tools/code send calls and record the response.
- › **Focuses on the business-logic layer** — not the visual layer an end-user sees.
- › **Faster & more stable than UI tests** — runs earlier, breaks less, pinpoints the failing contract.

Why API testing matters

Catches bugs early

below the UI, where logic lives — cheaper to fix.

Stable & fast

no flaky selectors; milliseconds per call.

Contract confidence

every consumer depends on the response shape.

Covers the NFRs

perf, security, reliability — invisible to the UI.

Functional vs Non-Functional requirements

Two questions about every API: does it do the job — and can it do it in production?

FUNCTIONAL (FR)

What the API must do.

- › “Return the current account balance.”
- › Right output for a given input.
- › Correct status, body & side-effects.
- › Makes the API work.

NON-FUNCTIONAL (NFR)

How well it must do it.

- › Fast · scalable · secure · available...
- › Quality under real conditions & load.
- › Measurable targets, not adjectives.
- › Makes the API production-ready.

Anatomy of an API call — what you assert on

Every API test inspects these parts of the request and the response.

request / response

```
# REQUEST
GET /accounts/42/balance HTTP/1.1
Host: api.eshop.vn
Authorization: Bearer <token>
X-Correlation-ID: c-789

# RESPONSE
HTTP/1.1 200 OK           <- status
Content-Type: application/json
X-RateLimit-Remaining: 99 <- headers
{ "accountId":42,
  "balance":1500000,      <- body
  "currency":"VND" }
```

ASSERT ON

- › **Status code** — 200 / 4xx / 5xx.
- › **Body values** — correct data.
- › **Schema** — field names & types.
- › **Headers** — auth, rate-limit, type.
- › **Latency** — response time.
- › **Side-effects** — state actually changed.

Where API tests sit — the pyramid

Many fast unit tests, a strong API/contract layer, few slow end-to-end tests.

E2E / UI — few — slow, flaky, whole-system

API / integration — the sweet spot — fast, stable, business logic

Unit — many — pure logic, milliseconds

API testing is where most functional AND non-functional checks live — this whole deck sits on this layer.

How to test an API — tools + environment

No GUI: you set up an environment, drive the API with a tool or code, and check the result.

TWO WAYS TO DRIVE IT

- › **Tooling** — Postman/Newman, Insomnia, Bruno, SoapUI, Hoppscotch.
- › **Code** — REST Assured (Java), supertest (JS), pytest+requests, Karate.
- › **Runner** — execute in CI on every push.

ENVIRONMENT SETUP

- › Configure DB + server per app requirements.
- › Seed known test data (fixtures).
- › Set base URL, tokens, secrets per env.
- › Smoke-call each endpoint before the suite.

Functional — does it return the right answer?

Requirement: “The API should return the current account balance.”

EXAMPLE — REST Assured

TEST TECHNIQUE

- › **Positive & negative** — valid id, unknown id, no token.
- › **Equivalence & BVA** — on inputs (id ranges, amounts).
- › **Assert status + schema + value** — not just 200.
- › **Verify side-effects** — read back state-changing calls.

```
// REST Assured
given().header("Authorization","Bearer "+tok)
.when().get("/accounts/42/balance")
.then()
    .statusCode(200)
    .body("balance", equalTo(1500000))
    .body("currency", equalTo("VND"))
    .body(matchesJsonSchema(balanceSchema));
// negative: unknown id -> 404 ; no token -> 401
```

ACCEPTANCE returns the correct current balance for a valid account; correct error status for invalid ones.

Contract testing — the schema IS the requirement

Consumers depend on the response shape; test the API against its OpenAPI/Pact contract.

EXAMPLE — Schemathesis / Pact

TEST TECHNIQUE

- › **Schema validation** — every response matches openapi.yaml.
- › **Consumer-driven (Pact)** — consumer's expectations verified on the provider.
- › **Spec-as-tests** — Dredd / Schemathesis run the spec.
- › **Fail CI on drift** — breaking change blocks the merge.

```
# Schemathesis – property tests from OpenAPI
schemathesis run openapi.yaml \
  --base-url https://api.eshop.vn

# every endpoint fuzzed & checked vs schema
GET /accounts/{id}/balance ... 200 ✓ schema
POST /transfers             ... 400 ✓ schema
# Pact: consumer contract verified on provider
```

Test the contract, not the whole system

Verify the request/response agreement between a service consumer and a provider — without a full end-to-end environment.

WHAT IT IS

- › **Consumer ↔ provider** — both sides agree on a shared Contract — the exact request & response shape.
- › **Consumer-driven (CDC)** — the consumer states what it needs; the provider must keep satisfying it.
- › **Tested independently** — each side runs its own fast test against the contract — no shared live env.
- › **The schema is the test** — renamed / removed fields, type or nullability changes are what it checks.

WHY IT MATTERS

- › **Built for microservices** — dozens of services stay compatible without one giant integration env.
- › **Decoupled & stable** — no flaky shared staging — teams deploy on their own cadence.
- › **Breaking changes, instantly** — a renamed field or missing param is flagged before it reaches production.
- › **Earlier than E2E** — catches integration breaks at build time, not during release testing.

WITHOUT IT everything passes in isolated local tests, then breaks at integration on a payload mismatch — the shared integration environment becomes the release bottleneck.

Contract vs functional vs end-to-end

Three questions, three techniques. Contract testing sits between fast functional checks and slow end-to-end integration.

Aspect	Functional API test	Contract test	End-to-end
Question	right value & status?	do both sides agree on the shape?	does the whole flow work?
Scope	one endpoint's behaviour	the schema between two services	many services + infra
Needs	the live API running	just the contract (pact) — a stub	all services deployed
Catches	wrong values, logic, status	breaking schema change (rename/type)	real wiring & integration bugs
Speed / cost	fast, cheap	very fast — unit-speed, isolated	slow · flaky · expensive
Tools	REST Assured · Postman	Pact · Spring Cloud Contract	Newman flows · full env

COMPLEMENT, NOT REPLACE contract tests catch breaking changes earlier and cheaper than E2E — but you still need functional tests for business correctness.

Pact — consumer-driven, broker-verified

The consumer records what it expects; the provider must keep satisfying that recorded contract.

EXAMPLE — Pact (consumer expectation)

HOW IT WORKS

- › **1 • Consumer test** — the mobile app's test declares the expected request & response → a pact file.
- › **2 • Broker** — the pact is published to a Pact Broker — the shared source of truth.
- › **3 • Provider verify** — the backend replays the pact in CI — it must satisfy every interaction.
- › **4 • Gate** — a breaking change fails verification → the build stops before deploy.

```
// CONSUMER (mobile) — expected interaction
provider
  .uponReceiving("a request for product 42")
  .withRequest({ method:"GET",
                 path:"/products/42" })
  .willRespondWith({ status:200, body:{
    id: 42,
    name: like("Shirt"),
    price: like(199000) }}); // a number

// PROVIDER renames price -> amount
=> verification FAILS: "price" is missing
```

BREAKING CHANGE CAUGHT renaming price → amount fails provider verification in CI — before the mobile app ever breaks in production.

THE CORE

9 Non-Functional Requirements

For each: what it means · the test technique · a concrete, runnable example · a measurable acceptance target.

1

Performance how fast it responds

NFR 1/9 How fast the API should respond under expected load.

EXAMPLE — k6

TEST TECHNIQUE

- › **Load testing** — measure latency while traffic runs.
- › **Percentiles, not averages** — p95 / p99 hide behind the mean.
- › **Set a threshold in CI** — fail the build if it regresses.
- › **Isolate** — warm cache, separate DB/network noise.

```
// k6 - load + latency threshold
export const options = {
  vus: 50, duration: "2m",
  thresholds:{ http_req_duration:["p(95)<2000"] }
};
export default () => {
  const r = http.get(`${B}/accounts/42/balance`);
  check(r,{ "200": x=>x.status===200 });
};
```

ACCEPTANCE (measurable) p95 response time < 2 s for 95% of requests at expected load.

NFR 2/9 Ability of the API to handle growth in users, data and transactions.

EXAMPLE — k6 stages

TEST TECHNIQUE

- › **Stress test** — ramp beyond normal; find the knee.
- › **Soak test** — hours at load; catch leaks/degradation.
- › **Spike test** — sudden surge, then recovery.
- › **Check horizontal scale** — more nodes → more throughput.

```
// ramp toward peak concurrency
export const options = {
  stages:[
    { duration:"2m", target: 500 },
    { duration:"5m", target: 5000 }, // peak
    { duration:"2m", target: 0 },
  ],
  thresholds:{ http_req_failed:["rate<0.01"] }
};
```

ACCEPTANCE (measurable) sustains 5,000 concurrent users at peak with error rate < 1%.

NFR 3/9 Protect the API and its data from unauthorized access and threats.

EXAMPLE — auth + BOLA

TEST TECHNIQUE

- › **AuthN / AuthZ tests** — missing/expired/wrong-role → 401/403.
- › **BOLA (API1:2023)** — user A must not read user B's object.
- › **Transport** — reject http://, require TLS 1.2+.
- › **DAST + fuzz** — OWASP ZAP; inputs as data, not code.

```
// BOLA: A requests B's account -> must 403
GET /accounts/999/balance
Authorization: Bearer <userA_token>
=> 403    (Broken Object Level Auth)

// auth matrix
no token    -> 401
expired     -> 401
wrong role  -> 403    (RBAC enforced)
```

ACCEPTANCE (measurable) OAuth 2.0 · TLS 1.2+ · RBAC · no BOLA — a user can never read another user's data.

4

Availability uptime & readiness

NFR 4/9 Uptime and readiness of the API for use.

TEST TECHNIQUE

- › **Health/readiness probe** — poll /health on an interval.
- › **Synthetic monitoring** — external probes measure real uptime.
- › **Failover / chaos** — kill a node; traffic must survive.
- › **Measure over a window** — uptime = ok / total probes.

EXAMPLE — health + SLO

```
// synthetic probe every 60s
GET /health -> 200 { "status":"UP" }

// SLO math over 30 days
uptime = ok_probes / total_probes
alert if uptime < 99.9%

// chaos: kill 1 of N nodes
=> requests fail over, still 200
```

ACCEPTANCE (measurable) ≥ 99.9% uptime (monthly) · health check green · survives a single-node failure.

NFR 5/9 How reliable the API is under normal and failure conditions.

TEST TECHNIQUE

- › **Idempotency test** — retry same op → no duplicate effect.
- › **Retry/timeout behaviour** — safe under network retries.
- › **Consistency after failure** — no half-applied transactions.
- › **Duplicate detection** — same key returns the first result.

EXAMPLE — idempotency

```
// first call creates transaction T1
POST /transfers
Idempotency-Key: abc-123  { amount:100 }
=> 201  (txn T1)

// network retry — SAME key
POST /transfers  Idempotency-Key: abc-123
=> 200, returns T1  (NO second transfer)
assert: exactly ONE transaction exists
```

ACCEPTANCE (measurable) retrying a request never creates a duplicate transaction.

Rate Limiting cap request volume

NFR 6/9 Limit the number of requests a client can make in a given time.

EXAMPLE — burst → 429

TEST TECHNIQUE

- › **Burst test** — fire past the limit; expect 429.
- › **Check headers** — Retry-After, X-RateLimit-Remaining.
- › **Per-client isolation** — one client's limit ≠ another's.
- › **Reset window** — allowed again after the window.

```
// same client, within 1 minute
for i in 1..101: GET /accounts/42/balance

// request 101:
HTTP 429 Too Many Requests
Retry-After: 30
X-RateLimit-Remaining: 0

// after window resets -> 200 again
```

ACCEPTANCE (measurable) a client cannot exceed 100 requests/minute; over-limit returns 429 + Retry-After.

Logging & Auditability track every call

NFR 7/9 Track and record API usage for monitoring and audit.

TEST TECHNIQUE

- › **Assert structured logs** — required fields on every request.
- › **Correlation-ID propagation** — one id across all services.
- › **No secrets in logs** — never log tokens / PII / PANs.
- › **Retention & queryable** — audit trail survives & is searchable.

EXAMPLE — log assertion

```
// request carries a correlation id
GET /accounts/42/balance
X-Correlation-ID: c-789

// exactly one structured log line:
{ ts, userId, correlationId:"c-789",
  method:"GET", path:"/accounts/42/balance",
  status:200, latencyMs:42 }
// assert: no token / no PII present
```

ACCEPTANCE (measurable) every request logs timestamp, user ID, correlation ID, payload, status, response time.

Error Handling fail clearly

NFR 8/9 How the API should handle errors and communicate them to callers.

TEST TECHNIQUE

- › **Negative testing** — one case per failure mode.
- › **Standardized error body** — code + message + traceId.
- › **Right HTTP status per failure** — 400/401/403/404/429/500.
- › **No leakage** — never return a stack trace on 500.

EXAMPLE — error contract

```
// not found
GET /accounts/NOPE/balance -> 404
{ "error": "ACCOUNT_NOT_FOUND",
  "message": "No account with id NOPE",
  "traceId": "c-789" }

// bad input->400 no token->401
// wrong role->403 over limit->429
// server fault->500 (generic, no stack)
```

ACCEPTANCE (measurable) standardized error codes + meaningful messages; correct HTTP status for every failure.

Maintainability easy to evolve

NFR 9/9 How easy it is to support, fix and enhance the API over time.

TEST TECHNIQUE

- › **Contract validation** — responses checked vs OpenAPI in CI.
- › **Versioning tests** — /v1 stays stable when /v2 ships.
- › **Breaking-change diff** — fail CI on incompatible spec change.
- › **Docs in sync** — spec ↔ implementation ↔ published docs.

EXAMPLE — versioning + spec

```
// OpenAPI is the source of truth
GET /v1/accounts/42/balance # v1 unchanged
GET /v2/accounts/42/balance # v2 adds field

// CI checks
validate responses vs openapi.yaml
oasdiff openapi_old openapi_new \
  --fail-on-breaking-changes
```

ACCEPTANCE (measurable) versioning strategy (/v1,/v2) · config documented · docs & spec always up to date.

Common HTTP status codes

The vocabulary of API responses — assert the right one for every case.

200	OK	request succeeded
400	Bad Request	malformed / invalid input
401	Unauthorized	missing / invalid auth
403	Forbidden	authenticated, not allowed (RBAC/BOLA)
404	Not Found	resource does not exist
429	Too Many Requests	rate limit exceeded
500	Internal Server Error	unhandled server fault

A good NFR is testable — or it's just a wish

Turn adjectives into numbers a test can check.

EVERY NFR MUST BE

- › **Measurable** — a number, not “fast”.
- › **Testable** — a check can pass/fail it.
- › **Clear & unambiguous** — one reading only.
- › **Aligned to business** — matches real needs.

ASK LIKE A BA

How fast? How secure?

How many users at peak?

What happens during failure?

How will it be monitored?

“respond in 2 s” beats “be fast”.

OWASP API Security Top 10 (2023)

The checklist your security tests should cover — BOLA leads for a reason.

API1

Broken Object Level Auth (BOLA)

API3

Broken Object Property Level Auth

API5

Broken Function Level Auth

API7

Server-Side Request Forgery (SSRF)

API9

Improper Inventory Management

API2

Broken Authentication

API4

Unrestricted Resource Consumption

API6

Unrestricted Access to Sensitive Flows

API8

Security Misconfiguration

API10

Unsafe Consumption of APIs

An API test strategy — layer the checks

Cheap deterministic checks first; expensive/slow ones later and less often.

Layer	Checks	Runs
Unit / component	logic, validators	every push
Functional / contract	status, schema, values, OpenAPI	every PR
Integration	cross-service flows, side-effects	every PR
Performance	load, stress, soak (k6/JMeter)	nightly / release
Security	authz, OWASP API Top 10, DAST	nightly / release

The API testing toolbox

Pick per job; most are free / open-source and CI-friendly.

Functional

Postman/Newman · REST Assured · supertest · Karate · Bruno

Performance

k6 · JMeter · Gatling · Locust

Mocking

WireMock · Prism · MockServer

Contract

OpenAPI/Swagger · Pact · Dredd · Schemathesis

Security

OWASP ZAP · Burp Suite · schemathesis (fuzz)

Monitoring

Prometheus + Grafana · synthetic uptime probes

More tools — design, code & contract

A fuller landscape. Most are free / open-source and CI-friendly.

Clients & design

Postman · Insomnia · Bruno · Hoppscotch · Apidog · Thunder Client · HTTPie · curl

Frameworks (code)

REST Assured (Java) · Karate DSL · supertest (JS) · pytest + requests · Playwright API · RestSharp (.NET) · Tavern · Newman CLI

Contract & spec

OpenAPI / Swagger · Pact · Dredd · Schemathesis · Spectral (lint) · Prism · Optic · oasdiff · Microcks

More tools — performance, security, mocks, monitoring

Performance / load

k6 · JMeter · Gatling · Locust · Artillery · Vegeta · wrk

Security (DAST / fuzz)

OWASP ZAP · Burp Suite · Schemathesis · StackHawk · 42Crunch · Akto · APIsec · Nuclei

Mocking / virtualization

WireMock · Mockoon · Prism · MockServer · Hoverfly · Microcks · Beeceptor

Monitoring / synthetic

Prometheus + Grafana · Checkly · Datadog Synthetics · Uptime Kuma · Postman Monitors · Runscope

The AI wave in API testing tools

New tools generate, self-heal and hunt — review their output like any other.

Tool	What it adds
Postbot — Postman	generate tests & assertions from a request
Apidog AI	spec → tests, AI assertions, full lifecycle
Keploy	record real traffic → auto-generate tests + mocks
Qodo (Cover)	AI test generation, finds coverage gaps
Katalon StudioAssist	AI authoring + self-healing tests
Akto	AI-driven API security testing
Parasoft SOAtest · Tricentis Tosca	agentic / Vision-AI enterprise suites

CAUTION AI-generated tests can assert the wrong thing or miss the real risk — a generated suite is a draft to review, not a verdict.

Choosing tools — and a free student stack

Match the tool to the layer; prefer open-source + CI-friendly; avoid tool sprawl.

HOW TO CHOOSE

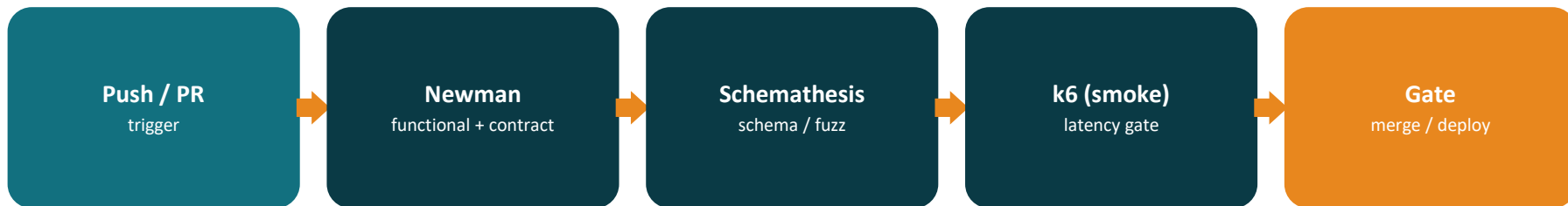
- › **Match the layer** — functional / contract / perf / security.
- › **Prefer open-source + CLI** — runs in CI, no licence wall.
- › **Keep tests portable** — don't lock into one vendor.
- › **Consolidate** — fewer tools beat tool sprawl.

A FREE STUDENT STACK

- › Postman / Newman — functional + contract
- › Schemathesis + Spectral — spec / fuzz
- › k6 — performance
- › OWASP ZAP — security
- › WireMock / Mockoon — mocks
- › Prometheus + Grafana — monitoring
- › Keploy — AI test-gen from traffic

API tests in the CI/CD pipeline

The whole harness runs on every change — functional gate always, heavy suites on schedule.



NIGHTLY / RELEASE full performance (stress/soak) + security (ZAP, OWASP API Top 10) — too slow for every PR.

RULE A broken functional or contract test blocks the merge. NFR regressions alert the team.

AI-assisted API testing

Use AI as a disciplined assistant — generate, fuzz, triage — then review every output.

Generate from spec

Feed the OpenAPI file; get positive + negative test skeletons to review.

guide

Property fuzzing

Schemathesis derives hundreds of edge cases from the schema automatically.

sensor

Failure triage

Paste a failing response + log; ask for ranked hypotheses + next checks.

sensor

TAKEAWAY

What makes an API production-ready

Functional requirements make the API work.

Non-functional requirements make it production-ready.

- Test EVERY requirement — FR and each NFR.
- Turn adjectives into measurable targets.
- Pair each requirement with a technique and a concrete, runnable example.

READING

OWASP API Security
Top 10 (2023)

k6 · JMeter docs

Pact / Schemathesis

Postman Learning Center
guru99.com/api-testing

Deliverable idea: for the EShop API, one test suite covering the functional requirement + all 9 NFRs with runnable examples.